

## Exception Handling

### Debugging Concepts and Techniques

**Debugging** is the process of identifying and rectifying errors in a program that leads to unexpected behavior. It is a critical component in programming that represents the most time-intensive and challenging phase of coding.

A typical debugging process includes the following:

- **Examining the error symptoms:** To successfully identify and understand the symptoms of the issue in the code, such as *crashes*, error messages, or unexpected behaviors.
- **Identifying the cause:** Once the symptoms are identified, the root cause of the problem should be taken care of. This can be done by tracing the code's execution and discovering specific lines creating the issue.
- **Fixing the error:** Once the issue is identified, necessary changes and corrections should be made to remove the problem and ensure that codes work correctly.

Here are some of debugging techniques to help master the process:

1. **Understanding the Problem:** It is vital to fully understand the problem being solved before making changes to the code. Without it, time will just be wasted, and solutions can be ineffective. It is advisable to reproduce the issue and gather as much information as possible.
2. **Backtracking:** This is also called **backward debugging** where it starts from the point the problem first began and works backward through the code to find out how and why it happened. This technique is useful when dealing with complex issues that are difficult to trace forward.
3. **Debugging Tools:** Tools for debugging can help give valuable insights into how the code functions, including variables and memory usage, that help save time. Examples include Chrome DevTools, Testsigma, and dbForge.
4. **Breakpoints and Stepping:** Setting breakpoints in the code where the issue might be hiding is a good practice. A **breakpoint** is the point in the code to temporarily stop the execution of the program to inspect the state of its functionality.

Then, using the **stepping** technique to manually move through each line of code to study variables and data structures for problems.

5. **Binary Search:** It is advisable to narrow the scope of the issues into manageable portions when faced with a complex problem. One way to do that is through **binary search** where the code is divided into halves and systematically narrowing the bug location. Commenting out code, using print statements, or isolating components can help isolate the problem area quickly.
6. **Rubber Ducking:** This is an analogy for explaining an object like a rubber duck to someone else to identify the issue. Articulating the problem out loud or written down forces critical thinking that can lead to discoveries of new insights and solutions.

7. **Log Analysis:** Placing log analysis statements in strategic areas of the code is a good idea to provide valuable information for debugging. Understanding the code's execution flow and the values of variables at different stages allows the debugger to identify the issue.
8. **Clustering Bugs:** Group error reports into classes of related bugs as they often share common causes or patterns. Fixing a bug from a class is likely to resolve other bugs or give insights into how to fix them.
9. **Take Breaks:** Debugging can be mentally taxing after hours of trying to resolve them. It is advisable to take breaks and step away from the code to clear the mind and allow the thoughts and ideas to simmer, leading to breakthroughs based on a fresh perspective.
10. **Take Notes:** Documenting the process and the solutions you used is an important technique, as it can be a valuable resource for future debugging challenges.

## Basic Exception Handling

An **exception** is an unexpected or error condition encountered in programming. Programs can generate several types of potential exceptions, such as the following:

- A program might issue a command to read a file from a disk, but the file does not exist there
- A program might attempt to write a disk, but the disk is full or unformatted
- A program might ask for user input, but the user enters an invalid data type
- A program might attempt to divide a value by 0
- A program might try to access an array with a subscript that is too large or too small

These are called *exceptions*, as they are not considered usual occurrences. An exception is not necessarily "bad" in nature. For instance, a company's profit might be stored in an `int`, but without a warning, the profit exceeds the highest value that an `int` can hold. It is not "bad", but it is still an exception.

**Exception handling** refers to the object-oriented techniques that manage or resolve such errors. If an unplanned exception occurs during a program's execution, it is called a **runtime error**; otherwise, if discovered during program compilation, it is called a **syntax error**.

There are two (2) basic classes of errors in Java:

- **Error Class** – represents more serious errors that a program usually might not recover from. For instance, an insufficient memory to execute a program. A program cannot recover from Error conditions on its own.
- **Exception Class** – represents less serious errors that indicate unusual conditions that arise while a program is running and from which the program can recover. For example, one type of Exception class error happens if a program uses an invalid array subscript value. The program can recover by assigning a valid value to the subscript variable.

Here is the Exception and Error class inheritance hierarchy.

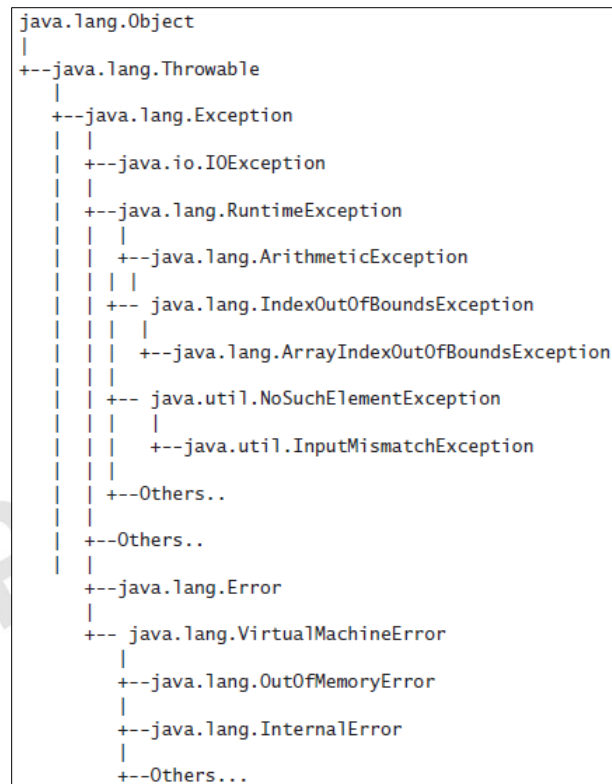


Figure 1. Exception and error classes. Retrieved from Farerel, J. (2023). Java programming, 10th edition. Cengage.

The Error and Exception classes descend from the Throwable class, which originates from the Object class. The Throwable class is considered the superclass for all errors and exceptions in the Java language.

Here are some exceptions and errors highlighted in Figure 1.

- **IOException** – the base class for exceptions thrown while accessing data from files, directories, and streams.
- **RuntimeException** – only detected during the execution of the application.
- **ArithmeticException** – raised whenever a wrong mathematical operation appears in the code during runtime.
- **IndexOutOfBoundsException** – thrown when an index used in arrays, lists, or strings is invalid.
- **ArrayIndexOutOfBoundsException** – indicates that an array has been accessed with an illegal index, either negative or greater than or equal to the size of the array.
- **NoSuchElementException** – indicates that the element being requested does not exist.
- **InputMismatchException** – occurs when the user does not provide the proper type of input or input is out of range.
- **VirtualMachineError** – indicates that the Java Virtual Machine is broken or has run out of resources necessary for it to continue operating.
- **OutOfMemoryError** – thrown when there is insufficient space to allocate an object.
- **InternalError** – indicates unexpected internal error has occurred in the Java Virtual Machine.

Given this example:

```
import java.util.Scanner;
public class Division {
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);
        int numerator, denominator, result;
        System.out.print("Enter numerator >> ");
        numerator = input.nextInt();
        System.out.print("Enter denominator >> ");
        denominator = input.nextInt();
        result = numerator / denominator;
        System.out.println(numerator + " / " + denominator + " = " + result);
    }
}
```

The sample Division class contains a single and small main() method that declares three (3) integers, prompts the user for values for two (2) of them, and calculates the value of the third integer by dividing the first two (2) values.

#### Execution #1:

```
Enter numerator >> 12
Enter denominator >> 4
12 / 4 = 3
```

#### Execution #2:

```
Enter numerator >> 12
Enter denominator >> 0
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Division.main(Division.java:12)
```

Based on the execution of the Division class, in Execution #1, the user enters two (2) usable values, and the program executes normally, while in Execution #2, the user enters 0 as the value for the denominator. Thus, an ArithmeticException message is displayed. Additionally, Execution #2 experienced a **crash**, meaning that it ended prematurely with an error.

#### Execution #3

```
Enter numerator >> 12
Enter denominator >> three
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:860)
    at java.base/java.util.Scanner.next(Scanner.java:1497)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2161)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2115)
    at Division.main(Division.java:11)
```

#### Execution #4

```
Enter numerator >> 12
Enter denominator >> 3.0
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:860)
    at java.base/java.util.Scanner.next(Scanner.java:1497)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2161)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2115)
    at Division.main(Division.java:11)
```

For Executions #3 and #4, the user entered noninteger data for the denominator: a string of characters (Execution #3) and a floating-point value (Execution #4), prompting an `InputMismatchException`.

The list of error messages, which shows each method called as the program ran after each attempted execution, is called a **stack trace history list** or **stack trace**.

### The try, catch, and finally Blocks

In OOP, programmers can “try” a procedure that might cause an error. A method that detects an error condition “throws an exception”, and if a block of code that processes the error is written, the block is said to “catch the exception”.

#### try block

This is placed when a programmer creates a segment of code in which something might go wrong. It is the block of code a programmer might attempt to execute while acknowledging that an exception might occur.

A try block consists of the following:

- The `try` keyword followed by a pair of curly braces
- Executable statements between the curly braces, including statements that might cause exceptions

#### catch block

To handle a thrown exception, one or more catch blocks can be added following a try block. A catch block is a segment of code that handles an exception that might be thrown by the try block that precedes it. The exception might be thrown automatically or explicitly when a `throw` statement is written.

A **throw statement** sends an `Exception` object out of a block or method so it can be handled elsewhere. A thrown `Exception` can be caught by a catch block.

A catch block consists of the following:

- The `catch` keyword followed by a pair of curly braces
- Between the parentheses, an `Exception` type and an identifier for an instance
- A pair of curly braces that contain statements that take actions to handle the error condition

The general format of a method that includes a try... catch pair.

```
returnType methodName(optional arguments) {
    // optional statements prior to code that is tried
    try {
        // statement or statements that might generate an exception
    }
    catch(Exception someException) {
        // actions to take if an exception occurs
    }
    // optional statements that occur whether the catch block executes or not
}
```

A catch block looks like a method named `catch()` that takes an argument that is some type of `Exception`. But it is not a method since it has no return type, and it cannot be called directly.

In the sample format, `someException` represents an object of the `Exception` class or any of its subclasses wherein the name can be any legal Java identifier. If an exception occurs during the execution of the try block, the exception is thrown and the statement in the catch block executes. Otherwise, the catch block does not execute. Lastly, any statements following the catch block execute normally.

Here is an example of applying the try... catch pair in the previous `Division` class program.

```
import java.util.Scanner;
public class DivisionMistakeCaught {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int numerator, denominator, result;
        System.out.print("Enter numerator >> ");
        numerator = input.nextInt();
        System.out.print("Enter denominator >> ");
        denominator = input.nextInt();
        try {
            result = numerator / denominator;
            System.out.println(numerator + " / " + denominator + " = " + result);
        }
        catch (ArithmeticException mistake) {
            System.out.println("Arithmetic exception was thrown and caught");
        }
        System.out.println("End of program");
    }
}
```

The `main()` method in the class contains a try block with code that attempts division. When an invalid integer division is attempted, an `ArithmeticException` is created and the catch block executes.

#### Execution #1

```
Enter numerator >> 15
Enter denominator >> 3
15 / 3 = 5
End of program
```

#### Execution #2

```
Enter numerator >> 15
Enter denominator >> 0
Arithmetic exception was thrown and caught
End of program
```

In Execution #1 of the `DivisionMistakeCaught` class, when the user enters a valid denominator, the complete try block executes, including the statement that displays the result, bypassing the catch block. In Execution #2, when the user enters 0 for the denominator, the try block is abandoned without displaying the division result, and the catch block executes, displaying the error message.

Lastly, whether the denominator is valid or not, the *End of program* message is displayed.

#### finally block

When actions must be performed at the end of a try... catch sequence, a finally block is used. The code written in the finally block executes whether or not the preceding try block identifies an `Exception`. Usually, this is used to perform cleanup tasks that must happen regardless of whether any `Exceptions` occurred or whether any `Exceptions` that occurred were caught.



The format of the try... catch... finally sequence

```
try {  
    // statements to try  
}  
catch(Exception e) {  
    // actions that occur if an exception was thrown  
}  
finally {  
    // actions that occur whether catch block executed or not  
}
```

When a finally block is included, there is an assurance that the finally statements will execute before the method is abandoned, even if the method concludes prematurely.

For example, here is a pseudocode that tries reading a file and handles an IOException.

```
try {  
    // Open the file  
    // Read the file  
    // Place the file data in an array  
    // Calculate an average from the data  
    // Display the average  
}  
catch(IOException e) {  
    // Issue an error message  
    // System exit  
}  
finally {  
    // If the file is open, close it  
}
```

By using the finally block, the file will be closed because the code in the finally block executes before the control returns to the operating system. It will execute no matter which outcome of the try block occurs.

#### References:

Bazhenov, P. (2023). *10 effective debugging techniques for developers*. [Web Article]. Retrieved on January 20, 2025, from <https://wearebrain.com/blog/10-effective-debugging-techniques-for-developers/>  
Pulok, M. (2024). *Learn advanced Java: Programming with beginners*.